Variation 3: The Fleet & Destination Planner

Deadline: December 28

The Scenario: "AeroData Dynamics"

Welcome to **AeroData Dynamics**, a global leader in aviation logistics and fleet management. You have just been hired as a **Junior Data Engineer**.

The company is currently undergoing a massive digital transformation. Historically, flight data, airport registries, and fleet information have been stored in disjointed CSV and JSON files (legacy relational exports). This fragmentation is making it impossible for the "Fleet Strategy Team" to analyze aircraft efficiency or track global destination statistics in real-time.

Your Task: Your Chief Technology Officer (CTO) has tasked you with migrating this legacy data into a **MongoDB Document Database**. You must move the data from flat files into a hierarchical format, clean the data types, and perform a series of advanced analytical queries to determine future fleet investments.

Task 1: Environment Setup:

1. **Download & Install: Install MongoDB Community Server and MongoDB Compass.**

Task 2: Import data into MongoDB:

1. You have been given **3 files:**
    a. **airports.csv:** A list of known airports
    b. **airlines.json:** A simple JSON array of airlines and their hubs.
    c. **flight_logs.csv:** A raw log of recent flights.
2. Create a **Localhost connection.**
3. Create a **database called "aviationDB".**
4. In the **database "aviationDB"** create **3 collections** called:
    a. **"airlines"**
    b. **"airports"**
    c. **"flight_logs"**
5. **Import the files "airports.csv"**, **"airlines.json"** and **"flight_logs.csv"** into their respective collections. **Use MongoDB Compass.**

6. **Crucial:** During import, you must intentionally set all data types to **string** (except for ObjectId). This is to simulate a common real-world scenario where data is imported incorrectly.

Task 3: Data Type Correction:

1. Write and execute **script** using **MongoDB Compass MongoDB Shell** that **fixes the datatypes** for **all the collections**.
2. For collection:
    a. **airlines:**
        i. Do not change any data types
    b. **airports:**
        i. Change the data types of:
            1. "lat" String → Double
            2. "lon" String → Double
    c. **flight_logs:**
        i. Change the data types of:
            1. "departure_time" String → Date
            2. "arrival_time" String → Date
            3. "passenger_count" String → Int32

Task 4: Denormalization (Build Analytics Collection):

Your flight_logs collection is *normalized*. For fast analysis, we need to *denormalize* (embed) related data.

Write and execute **script** with an aggregation pipeline to build a new collection called "**flights_enhanced**" using **MongoDB Compass MongoDB Shell**.

1. Look up and embed related data:
    a. Use the dest_iata (from flight_logs) to find the matching airport from the airports collection (iata_code).
    b. Embed this data as a new field called dest_airport_details.
2. Calculate a new field:
    a. Create a new field named flight_duration_minutes.
    b. This field's value should be calculated by subtracting the departure_time from the arrival_time.
    c. The result should be in minutes.
3. The final stage of your pipeline must output all the new, transformed documents into a new collection named "**flights_enhanced**".

Task 5: Advanced Aggregation Queries:

Task 5.1: European Destination Analysis

1. **Task:** Track passenger flow into key European nations.
2. **Output:** Filter for flights where the destination country is "**France**" OR "**Germany**". Group by the **Destination City** and calculate the **Average Passenger Count**.
3. **Sort:** By **Average Passenger Count** in **descending** order. Limit output to **4**.

Task 5.2: Under-Utilized Long Hauls

1. **Task:** Identify inefficient long-distance routes.
2. **Output:** Find flights where **flight_duration_minutes** is **greater than 400** AND **passenger_count** is **less than 100**. Show the **Flight ID**, **Destination Country**, and **Passenger Count**.
3. By **Passenger Count** in **ascending** order. Limit output to **7**.

Task 5.3: Aircraft Range Tier

1. **Task:** Categorize aircraft by their average operational range.
2. **Output:** Calculate the **Average Duration** for each **aircraft_model**. Filter to show only models with an average duration **less than or equal to 650** minutes.
3. **Sort:** By **Average Duration** in **descending** order. Limit output to **10**.

Task 5.4: Busy Route Identification

1. **Task:** Find the specific routes (Origin -> Destination) with the highest frequency.
2. **Output:** Group by **dest_iata** (Destination Code). Calculate the **Total Flights** (count). Filter to ensure Total Flights is **greater than or equal to 100**.
3. **Sort:** By **Total Flights** in **ascending** order (to see the "just busy enough" routes). Limit output to **5**.

Remember to take all your created scripts and queries and put them in a single .js file so that the instructor can load a single .js file to check if the task is completed correctly.